



BlockStreet 2

Security Assessment

CertiK Assessed on Jan 30th, 2026





Certik Assessed on Jan 30th, 2026

BlockStreet 2

The security assessment was prepared by Certik.

Executive Summary

TYPES

Governance

ECOSYSTEM

Binance Smart Chain
(BSC)

METHODS

Formal Verification, Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Preliminary comments published on 11/26/2025

Final report published on 01/30/2026

Vulnerability Summary



4

Total Findings

2

Resolved

0

Partially Resolved

2

Acknowledged

0

Declined

1 Centralization

1 Acknowledged



Centralization findings highlight privileged roles & functions and their capabilities, or instances where the project takes custody of users' assets.

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

0 Major

Major risks may include logical errors that, under specific circumstances, could result in fund losses or loss of project control.

0 Medium

Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

2 Minor

2 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

1 Informational

1 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | BLOCKSTREET 2

Summary

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

Review Notes

[Overview](#)

[External Dependencies](#)

[Privileged Functions](#)

Findings

[BL2-05 : Initial Token Distribution](#)

[BL2-01 : Getter Misalignment With Actual Vesting Schedule](#)

[BL2-02 : Invalid Vesting Start Time Check Leads To Misconfigured Vesting Schedule](#)

[BL2-04 : Redundant Schedule Index Check Leads To Minor Gas Waste And Confusion](#)

Formal Verification

[Considered Functions And Scope](#)

[Verification Results](#)

Appendix

Disclaimer

CODEBASE | BLOCKSTREET 2

Repository

<https://github.com/BlockStreet-finance/bsb-token/tree/293962e0432bf028eb6fcbf415bb162d7a60fc70>

<https://github.com/BlockStreet-finance/bsb-token/tree/d9b7b55114b912c5afb909c3f4b4ae57d536528e>

AUDIT SCOPE | BLOCKSTREET 2

mainnet

 src/BlockStreetToken.sol

BlockStreet-finance/bsb-token

 src/BSTTokenMultiVesting.sol

 src/BSTTokenMultiVesting.sol

APPROACH & METHODS | BLOCKSTREET 2

This audit was conducted for BlockStreet to evaluate the security and correctness of the smart contracts associated with the BlockStreet 2 project. The assessment included a comprehensive review of the in-scope smart contracts. The audit was performed using a combination of Static Analysis, Formal Verification, and Manual Review.

The review process emphasized the following areas:

- Architecture review and threat modeling to understand systemic risks and identify design-level flaws.
- Identification of vulnerabilities through both common and edge-case attack vectors.
- Manual verification of contract logic to ensure alignment with intended design and business requirements.
- Dynamic testing to validate runtime behavior and assess execution risks.
- Assessment of code quality and maintainability, including adherence to current best practices and industry standards.

The audit resulted in findings categorized across multiple severity levels, from informational to critical. To enhance the project's security and long-term robustness, we recommend addressing the identified issues and considering the following general improvements:

- Improve code readability and maintainability by adopting a clean architectural pattern and modular design.
- Strengthen testing coverage, including unit and integration tests for key functionalities and edge cases.
- Maintain meaningful inline comments and documentations.
- Implement clear and transparent documentation for privileged roles and sensitive protocol operations.
- Regularly review and simulate contract behavior against newly emerging attack vectors.

REVIEW NOTES | BLOCKSTREET 2

Overview

BlockStreetToken (BSB) — simple ERC20 token (OpenZeppelin) with fixed supply of 1,000,000,000 BSB minted to deployer.

The **BSTTokenMultiVesting** contract implements multi-schedule token vesting for ERC20 tokens. Key features:

- multiple vesting schedules per beneficiary (each schedule: cliff + N * 30-day periods),
- configurable cliff and per-period amount,
- owner-managed schedule creation, batch creation, update and removal (updates/removals allowed only before vesting starts),
- beneficiaries can transfer all their schedules to a new address,
- users can claim per-schedule or claim all available tokens,
- many read/projection/export utilities and a health check.

The contract aims to be flexible and query-friendly, but several correctness, safety, and UX issues were found (detailed below).

External Dependencies

- OpenZeppelin `IERC20`, `Ownable`, `ReentrancyGuard` (imports shown).
- The token used must be a standard ERC20 (transfer semantics, no unexpected transfer fees, and ideally returning `bool`).

Privileged Functions

Functions callable only by the owner that materially affect funds and the schedule state:

- `addSchedule`, `batchAddSchedules`, `_addScheduleInternal` — create allocations (increase `totalAllocatedAmount`).
- `updateSchedule` — change schedule amounts before vesting starts (updates `totalAllocatedAmount`).
- `removeSchedule` — deactivate schedules before vesting starts (decreases `totalAllocatedAmount`).
- `setStartTime`, `startVesting` — control vesting start and enable claiming.

Because the owner can change allocations and start vesting, owner keys must be secured (multisig/timelock), and all owner actions should be audited/monitored.

FINDINGS | BLOCKSTREET 2



4

Total Findings

0

Critical

1

Centralization

0

Major

0

Medium

2

Minor

1

Informational

This report has been prepared for BlockStreet to identify potential vulnerabilities and security issues within the reviewed codebase. During the course of the audit, a total of 4 issues were identified. Leveraging a combination of Static Analysis, Formal Verification & Manual Review the following findings were uncovered:

ID	Title	Category	Severity	Status
BL2-05	Initial Token Distribution	Centralization	Centralization	● Acknowledged
BL2-01	Getter Misalignment With Actual Vesting Schedule	Incorrect Calculation	Minor	● Resolved
BL2-02	Invalid Vesting Start Time Check Leads To Misconfigured Vesting Schedule	Logical Issue	Minor	● Resolved
BL2-04	Redundant Schedule Index Check Leads To Minor Gas Waste And Confusion	Coding Style	Informational	● Acknowledged

BL2-05 | Initial Token Distribution

Category	Severity	Location	Status
Centralization	● Centralization	src/BlockStreetToken.sol (0xdb6ba5d510f114f9b2ea08bea7d30e32eee33411): 16-17	● Acknowledged

Description

All of the BSB tokens are sent to the contract deployer or one or several externally-owned account (EOA) addresses. This is a centralization risk because the deployer or the owner(s) of the EOAs can distribute tokens without obtaining the consensus of the community. Any compromise to these addresses may allow a hacker to steal and sell tokens on the market, resulting in severe damage to the project.

Recommendation

It's recommended that the team publicly disclose a clear and detailed initial token distribution plan accessible to the community. The audit team recommends that undistributed tokens be secured in a multi-signature wallet to minimize the risks associated with single-key control. Access to private keys related to critical accounts, including the deployer and EOAs, should be tightly controlled and monitored. Consider implementing time-locked or vesting mechanisms for significant token allocations to promote project stability and long-term alignment. To further enhance trust and accountability, the team should review options for undergoing third-party KYC verification.

Alleviation

[BlockStreet, 01/29/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

BL2-01 | Getter Misalignment With Actual Vesting Schedule

Category	Severity	Location	Status
Incorrect Calculation	● Minor	src/BSTTokenMultiVesting.sol (293962e0432bf028eb6fcbf415bb162d7a60fc70): 563~573, 681~772	● Resolved

Description

calculateVestedAmount explicitly unlocks the first period immediately after the cliff by adding one to the completed-period count, which matches the deployment guide's description of "cliff ends → first tranche releases instantly."

```
if (currentTime < startTime + schedule.cliffDuration) {
    return 0;
}

// Calculate periods completed after cliff (first period unlocks immediately
after cliff)
uint256 timeAfterCliff = currentTime - (startTime + schedule.cliffDuration);
uint256 periodsCompleted = (timeAfterCliff / PERIOD_DURATION) + 1; // @audit
// Fully vested
if (periodsCompleted >= schedule.numberOfPeriods) {
    return schedule.totalAmount;
}

// Calculate vested amount based on completed periods
return periodsCompleted * schedule.amountPerPeriod;
}
```

However, both `getProjectedVestingAt` and `getUserProjectedVestingAt` omit the +1 offset after the cliff, so they report zero vested tokens at the cliff boundary while the claim functions would already release `amountPerPeriod`. Dashboards relying on these views will consistently under-display the available balance.

```
if (timestamp < cliffEnd) {
    continue;
}
uint256 timeAfterCliff = timestamp - cliffEnd;
uint256 periodsCompleted = timeAfterCliff / PERIOD_DURATION;
if (periodsCompleted >= schedule.numberOfPeriods) {
    projectedVested += schedule.totalAmount;
} else {
    projectedVested += periodsCompleted * schedule.amountPerPeriod;
}
```

Additionally, `getScheduleStateCounts` function that surface “vesting end” details add `numberOfPeriods * PERIOD_DURATION` to the cliff instead of `(numberOfPeriods - 1) * PERIOD_DURATION`, even though vesting logic completes after the (n-1)th interval.

```
uint256 vestingEnd = cliffEnd + (schedule.numberOfPeriods *  
PERIOD_DURATION);  
  
if (currentTime < cliffEnd) {  
    inCliff++;  
} else if (currentTime < vestingEnd) {  
    vesting++;  
} else {  
    completed++;  
}
```

Recommendation

Consider aligning every getter and reporting path with the canonical vesting math: add the immediate post-cliff tranche when computing projected vested/claimable amounts, and set vesting completion timestamps to `cliffEnd + ((numberOfPeriods - 1) * PERIOD_DURATION)`

Alleviation

[BlockStreet, 01/29/2026]: The team heeded the advice and resolved the issue in commit [d9b7b55114b912c5afb909c3f4b4ae57d536528e](#)

BL2-02 | Invalid Vesting Start Time Check Leads To Misconfigured Vesting Schedule

Category	Severity	Location	Status
Logical Issue	Minor	src/BSTTokenMultiVesting.sol (293962e0432bf028eb6fcbf415bb162d7a60fc70): 228~232	Resolved

Description

The function `setStartTime` is designed to allow the contract owner to define the vesting schedule's starting timestamp before vesting begins.

```
228     function setStartTime(uint256 _startTime) external onlyOwner whenNotStarted {
229         require(_startTime > 0, "Invalid start time");
230         startTime = _startTime;
231         emit StartTimeSet(_startTime);
232     }
```

However, the function only checks that `_startTime` is greater than zero, without validating that it is greater than the current block timestamp. This allows the owner to mistakenly (or maliciously) set a vesting start time in the past, which may cause the vesting logic relying on `startTime` to behave incorrectly, such as instantly unlocking tokens or skipping part of the vesting period.

Recommendation

The contract should enforce that the provided start time is strictly greater than the current block timestamp. Updating the validation to `require(_startTime > block.timestamp, "Start time must be in the future")` ensures that vesting schedules cannot be misconfigured or manipulated.

Alleviation

[BlockStreet, 01/29/2026]: The team heeded the advice and resolved the issue by requiring it greater than the current time in commit [d9b7b55114b912c5afb909c3f4b4ae57d536528e](#)

BL2-04 | Redundant Schedule Index Check Leads To Minor Gas Waste And Confusion

Category	Severity	Location	Status
Coding Style	● Informational	src/BSTTokenMultiVesting.sol (293962e0432bf028eb6fcbf415bb162d7a60fc70): 376~379	● Acknowledged

Description

```
376         uint256 vested = calculateVestedAmount(user, scheduleIndex);
377         if (vested == 0 || scheduleIndex >= userSchedules[user].length) {
378             return 0;
379         }
```

1. The function `calculateClaimableAmount` computes how much a user can claim for a vesting schedule by first calling `calculateVestedAmount(user, scheduleIndex)` and then checking `if (vested == 0 || scheduleIndex >= userSchedules[user].length)`.
2. The condition `scheduleIndex >= userSchedules[user].length` is redundant because `calculateVestedAmount` already returns `0` when `scheduleIndex` is out of bounds; calling the expensive helper before bounds-checking wastes gas and makes the code confusing.
3. The impact is small: unnecessary gas is consumed (especially for repeated or external calls), and future readers/maintainers may misinterpret the intent, increasing the chance of subtle bugs when the functions are modified.

Recommendation

Validate the `scheduleIndex` before calling `calculateVestedAmount`, and remove the redundant clause. This saves gas and makes the control flow explicit.

Alleviation

[BlockStreet, 01/29/2026]: The team acknowledged the issue and decided not to implement the recommended change in the current engagement.

FORMAL VERIFICATION | BLOCKSTREET 2

Formal guarantees about the behavior of smart contracts can be obtained by reasoning about properties relating to the entire contract (e.g. contract invariants) or to specific functions of the contract. Once such properties are proven to be valid, they guarantee that the contract behaves as specified by the property. As part of this audit, we applied formal verification to prove that important functions in the smart contracts adhere to their expected behaviors.

Considered Functions And Scope

In the following, we provide a description of the properties that have been used in this audit. They are grouped according to the type of contract they apply to.

Verification of Standard Ownable Properties

We verified *partial* properties of the public interfaces of those token contracts that implement the Ownable interface. This involves:

- function `owner` that returns the current owner,
- functions `renounceOwnership` that removes ownership,
- function `transferOwnership` that transfers the ownership to a new owner.

The properties that were considered within the scope of this audit are as follows:

Property Name	Title
ownable-renounceownership-correct	Ownership is Removed
ownable-transferownership-correct	Ownership is Transferred
ownable-owner-succeed-normal	<code>owner</code> Always Succeeds
ownable-renounce-ownership-is-permanent	Once Renounced, Ownership Cannot be Regained

Verification of ERC-20 Compliance

We verified properties of the public interface of those token contracts that implement the ERC-20 interface. This covers

- Functions `transfer` and `transferFrom` that are widely used for token transfers,
- functions `approve` and `allowance` that enable the owner of an account to delegate a certain subset of her tokens to another account (i.e. to grant an allowance), and
- the functions `balanceOf` and `totalSupply`, which are verified to correctly reflect the internal state of the contract.

The properties that were considered within the scope of this audit are as follows (note that overflow properties were excluded from the verification):

Property Name	Title
erc20-transfer-correct-amount	<code>transfer</code> Transfers the Correct Amount in Transfers
erc20-transferfrom-correct-allowance	<code>transferFrom</code> Updated the Allowance Correctly
erc20-transferfrom-correct-amount	<code>transferFrom</code> Transfers the Correct Amount in Transfers
erc20-approve-correct-amount	<code>approve</code> Updates the Approval Mapping Correctly
erc20-allowance-succeed-always	<code>allowance</code> Always Succeeds
erc20-transfer-false	If <code>transfer</code> Returns <code>false</code> , the Contract State Is Not Changed
erc20-allowance-correct-value	<code>allowance</code> Returns Correct Value
erc20-transferfrom-fail-exceed-allowance	<code>transferFrom</code> Fails if the Requested Amount Exceeds the Available Allowance
erc20-transfer-never-return-false	<code>transfer</code> Never Returns <code>false</code>
erc20-transferfrom-revert-zero-argument	<code>transferFrom</code> Fails for Transfers with Zero Address Arguments
erc20-transfer-revert-zero	<code>transfer</code> Prevents Transfers to the Zero Address
erc20-transferfrom-fail-exceed-balance	<code>transferFrom</code> Fails if the Requested Amount Exceeds the Available Balance
erc20-transfer-exceed-balance	<code>transfer</code> Fails if Requested Amount Exceeds Available Balance
erc20-transferfrom-false	If <code>transferFrom</code> Returns <code>false</code> , the Contract's State Is Unchanged
erc20-approve-never-return-false	<code>approve</code> Never Returns <code>false</code>
erc20-approve-succeed-normal	<code>approve</code> Succeeds for Valid Inputs
erc20-transferfrom-never-return-false	<code>transferFrom</code> Never Returns <code>false</code>
erc20-balanceof-change-state	<code>balanceOf</code> Does Not Change the Contract's State
erc20-balanceof-succeed-always	<code>balanceOf</code> Always Succeeds
erc20-totalsupply-change-state	<code>totalSupply</code> Does Not Change the Contract's State
erc20-allowance-change-state	<code>allowance</code> Does Not Change the Contract's State
erc20-approve-revert-zero	<code>approve</code> Prevents Approvals For the Zero Address

Property Name	Title
erc20-approve-false	If <code>approve</code> Returns <code>false</code> , the Contract's State Is Unchanged
erc20-balanceof-correct-value	<code>balanceOf</code> Returns the Correct Value
erc20-totalsupply-correct-value	<code>totalSupply</code> Returns the Value of the Corresponding State Variable
erc20-totalsupply-succeed-always	<code>totalSupply</code> Always Succeeds

Verification Results

For the following contracts, formal verification established that each of the properties that were in scope of this audit (see scope) are valid:

Detailed Results For Contract BlockStreetToken (src/BlockStreetToken.sol) In Commit 0xdb6ba5d510f114f9b2ea08bea7d30e32eee33411

Verification of ERC-20 Compliance

Detailed Results for Function `transfer`

Property Name	Final Result	Remarks
erc20-transfer-correct-amount	● True	
erc20-transfer-false	● True	
erc20-transfer-never-return-false	● True	
erc20-transfer-revert-zero	● True	
erc20-transfer-exceed-balance	● True	

Detailed Results for Function `transferFrom`

Property Name	Final Result	Remarks
erc20-transferfrom-correct-allowance	● True	
erc20-transferfrom-correct-amount	● True	
erc20-transferfrom-fail-exceed-allowance	● True	
erc20-transferfrom-revert-zero-argument	● True	
erc20-transferfrom-fail-exceed-balance	● True	
erc20-transferfrom-false	● True	
erc20-transferfrom-never-return-false	● True	

Detailed Results for Function `approve`

Property Name	Final Result	Remarks
erc20-approve-correct-amount	● True	
erc20-approve-never-return-false	● True	
erc20-approve-succeed-normal	● True	
erc20-approve-revert-zero	● True	
erc20-approve-false	● True	

Detailed Results for Function `allowance`

Property Name	Final Result	Remarks
erc20-allowance-succeed-always	● True	
erc20-allowance-correct-value	● True	
erc20-allowance-change-state	● True	

Detailed Results for Function `balanceOf`

Property Name	Final Result	Remarks
erc20-balanceof-change-state	● True	
erc20-balanceof-succeed-always	● True	
erc20-balanceof-correct-value	● True	

Detailed Results for Function `totalSupply`

Property Name	Final Result	Remarks
erc20-totalsupply-change-state	● True	
erc20-totalsupply-correct-value	● True	
erc20-totalsupply-succeed-always	● True	

In the remainder of this section, we list all contracts where formal verification of at least one property was not successful. There are several reasons why this could happen:

- False: The property is violated by the project.
- Inconclusive: The proof engine cannot prove or disprove the property due to timeouts or exceptions.
- Inapplicable: The property does not apply to the project.

Detailed Results For Contract BSTTokenMultiVesting (src/BSTTokenMultiVesting.sol) In Commit d9b7b55114b912c5afb909c3f4b4ae57d536528e**Verification of Standard Ownable Properties**Detailed Results for Function `renounceOwnership`

Property Name	Final Result	Remarks
ownable-renounceownership-correct	● True	
ownable-renounce-ownership-is-permanent	● Inconclusive	

Detailed Results for Function `transferOwnership`

Property Name	Final Result	Remarks
ownable-transferownership-correct	● True	

Detailed Results for Function `owner`

Property Name	Final Result	Remarks
ownable-owner-succeed-normal	● True	

Detailed Results For Contract BSTTokenMultiVesting (src/BSTTokenMultiVesting.sol) In Commit 293962e0432bf028eb6fcbf415bb162d7a60fc70

Verification of Standard Ownable Properties

Detailed Results for Function `renounceOwnership`

Property Name	Final Result	Remarks
ownable-renounceownership-correct	● True	
ownable-renounce-ownership-is-permanent	● Inconclusive	

Detailed Results for Function `owner`

Property Name	Final Result	Remarks
ownable-owner-succeed-normal	● True	

Detailed Results for Function `transferOwnership`

Property Name	Final Result	Remarks
ownable-transferownership-correct	● True	

APPENDIX | BLOCKSTREET 2

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Incorrect Calculation	Incorrect Calculation findings are about issues in numeric computation such as rounding errors, overflows, out-of-bounds and any computation that is not intended.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

Details on Formal Verification

Some Solidity smart contracts from this project have been formally verified. Each such contract was compiled into a mathematical model that reflects all its possible behaviors with respect to the property. The model takes into account the semantics of the Solidity instructions found in the contract. All verification results that we report are based on that model.

The following assumptions and simplifications apply to our model:

- Certain low-level calls and inline assembly are not supported and may lead to a contract not being formally verified.
- We model the semantics of the Solidity source code and not the semantics of the EVM bytecode in a compiled contract.

Formalism for property specifications

All properties are expressed in a behavioral interface specification language that CertiK has developed for Solidity, which allows us to specify the behavior of each function in terms of the contract state and its parameters and return values, as well as contract properties that are maintained by every observable state transition. Observable state transitions occur when the contract's external interface is invoked and the invocation does not revert, and when the contract's Ether balance is changed by the EVM due to another contract's "self-destruct" invocation. The specification language has the usual Boolean connectives, as well as the operator `\old` (used to denote the state of a variable before a state transition), and several types of specification clause:

Apart from the Boolean connectives and the modal operators "always" (written `[]`) and "eventually" (written `<>`), we use the following predicates to reason about the validity of atomic propositions. They are evaluated on the contract's state whenever a discrete time step occurs:

- `requires [cond]` - the condition `cond`, which refers to a function's parameters, return values, and contract state variables, must hold when a function is invoked in order for it to exhibit a specified behavior.

- `ensures [cond]` - the condition `cond`, which refers to a function's parameters, return values, and both `\old` and current contract state variables, is guaranteed to hold when a function returns if the corresponding requires condition held when it was invoked.
- `invariant [cond]` - the condition `cond`, which refers only to contract state variables, is guaranteed to hold at every observable contract state.
- `constraint [cond]` - the condition `cond`, which refers to both `\old` and current contract state variables, is guaranteed to hold at every observable contract state except for the initial state after construction (because there is no previous state); constraints are used to restrict how contract state can change over time.

Description of the Analyzed ERC-20 Properties

Properties related to function `transfer`

erc20-transfer-correct-amount

All non-reverting invocations of `transfer(recipient, amount)` that return `true` must subtract the value in `amount` from the balance of `msg.sender` and add the same value to the balance of the `recipient` address.

Specification:

```
requires recipient != msg.sender;
requires balanceOf(recipient) + amount <= type(uint256).max;
ensures \result ==> balanceOf(recipient) == \old(balanceOf(recipient) + amount)
  && balanceOf(msg.sender) == \old(balanceOf(msg.sender) - amount);
  also
requires recipient == msg.sender;
ensures \result ==> balanceOf(msg.sender) == \old(balanceOf(msg.sender));
```

erc20-transfer-exceed-balance

Any transfer of an amount of tokens that exceeds the balance of `msg.sender` must fail.

Specification:

```
requires amount > balanceOf(msg.sender);
ensures !\result;
```

erc20-transfer-false

If the `transfer` function in contract `BlockStreetToken` fails by returning `false`, it must undo all state changes it incurred before returning to the caller.

Specification:

```
ensures !\result ==> \assigned (\nothing);
```

erc20-transfer-never-return-false

The transfer function must never return `false` to signal a failure.

Specification:

```
ensures \result;
```

erc20-transfer-revert-zero

Any call of the form `transfer(recipient, amount)` must fail if the recipient address is the zero address.

Specification:

```
ensures \old(recipient) == address(0) ==> !\result;
```

Properties related to function `transferFrom`**erc20-transferfrom-correct-allowance**

All non-reverting invocations of `transferFrom(from, dest, amount)` that return `true` must decrease the allowance for address `msg.sender` over address `from` by the value in `amount`.

Specification:

```
ensures \result ==> allowance(\old(sender), msg.sender) == \old(allowance(sender,
msg.sender)) - \old(amount)
          || (allowance(\old(sender), msg.sender) == \old(allowance(sender,
msg.sender)) && \old(allowance(sender, msg.sender)) == type(uint256).max);
```

erc20-transferfrom-correct-amount

All invocations of `transferFrom(from, dest, amount)` that succeed and that return `true` subtract the value in `amount` from the balance of address `from` and add the same value to the balance of address `dest`.

Specification:

```
requires recipient != sender;
requires balanceOf(recipient) + amount <= type(uint256).max;
ensures \result ==> balanceOf(\old(recipient)) == \old(balanceOf(recipient) +
amount)
          && balanceOf(\old(sender)) == \old(balanceOf(sender) - amount);
also
requires recipient == sender;
ensures \result ==> balanceOf(\old(recipient)) == \old(balanceOf(recipient));
```

erc20-transferfrom-fail-exceed-allowance

Any call of the form `transferFrom(from, dest, amount)` with a value for `amount` that exceeds the allowance of address `msg.sender` must fail.

Specification:

```
requires msg.sender != sender;  
requires amount > allowance(sender, msg.sender);  
ensures !\result;
```

erc20-transferfrom-fail-exceed-balance

Any call of the form `transferFrom(from, dest, amount)` with a value for `amount` that exceeds the balance of address `from` must fail.

Specification:

```
requires amount > balanceOf(sender);  
ensures !\result;
```

erc20-transferfrom-false

If `transferFrom` returns `false` to signal a failure, it must undo all incurred state changes before returning to the caller.

Specification:

```
ensures !\result ==> \assigned (\nothing);
```

erc20-transferfrom-never-return-false

The `transferFrom` function must never return `false`.

Specification:

```
ensures \result;
```

erc20-transferfrom-revert-zero-argument

All calls of the form `transferFrom(from, dest, amount)` must fail for transfers from or to the zero address.

Specification:

```
ensures \old(sender) == address(0) ==> !\result;  
also  
ensures \old(recipient) == address(0) ==> !\result;
```

Properties related to function `approve`**erc20-approve-correct-amount**

All non-reverting calls of the form `approve(spender, amount)` that return `true` must correctly update the allowance mapping according to the address `msg.sender` and the values of `spender` and `amount`.

Specification:

```
requires spender != address(0);
ensures \result ==> allowance(msg.sender, \old(spender)) == \old(amount);
```

erc20-approve-false

If function `approve` returns `false` to signal a failure, it must undo all state changes that it incurred before returning to the caller.

Specification:

```
ensures !\result ==> \assigned (\nothing);
```

erc20-approve-never-return-false

The function `approve` must never returns `false`.

Specification:

```
ensures \result;
```

erc20-approve-revert-zero

All calls of the form `approve(spender, amount)` must fail if the address in `spender` is the zero address.

Specification:

```
ensures \old(spender) == address(0) ==> !\result;
```

erc20-approve-succeed-normal

All calls of the form `approve(spender, amount)` must succeed, if

- the address in `spender` is not the zero address and
- the execution does not run out of gas.

Specification:

```
requires spender != address(0);
ensures \result;
reverts_only_when false;
```

Properties related to function `allowance`

erc20-allowance-change-state

Function `allowance` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc20-allowance-correct-value

Invocations of `allowance(owner, spender)` must return the allowance that address `spender` has over tokens held by address `owner`.

Specification:

```
ensures \result == allowance(\old(owner), \old(spender));
```

erc20-allowance-succeed-always

Function `allowance` must always succeed, assuming that its execution does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `balanceOf`

erc20-balanceof-change-state

Function `balanceOf` must not change any of the contract's state variables.

Specification:

```
assignable \nothing;
```

erc20-balanceof-correct-value

Invocations of `balanceOf(owner)` must return the value that is held in the contract's balance mapping for address `owner`.

Specification:

```
ensures \result == balanceOf(\old(account));
```

erc20-balanceof-succeed-always

Function `balanceOf` must always succeed if it does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `totalSupply`

erc20-totalsupply-change-state

The `totalSupply` function in contract `BlockStreetToken` must not change any state variables.

Specification:

```
assignable \nothing;
```

erc20-totalsupply-correct-value

The `totalSupply` function must return the value that is held in the corresponding state variable of contract `BlockStreetToken`.

Specification:

```
ensures \result == totalSupply();
```

erc20-totalsupply-succeed-always

The function `totalSupply` must always succeeds, assuming that its execution does not run out of gas.

Specification:

```
reverts_only_when false;
```

Description of the Analyzed Ownable Properties

Properties related to function `renounceOwnership`

ownable-renounce-ownership-is-permanent

The contract must prohibit regaining of ownership once it has been renounced.

Specification:

```
constraint \old(owner()) == address(0) ==> owner() == address(0);
```

ownable-renounceownership-correct

Invocations of `renounceOwnership()` must set ownership to `address(0)`.

Specification:

```
ensures this.owner() == address(0);
```

Properties related to function `transferOwnership`

ownable-transferownership-correct

Invocations of `transferOwnership(newOwner)` must transfer the ownership to the `newOwner`.

Specification:

```
ensures this.owner() == newOwner;
```

Properties related to function `owner`

ownable-owner-succeed-normal

Function `owner` must always succeed if it does not run out of gas.

Specification:

```
reverts_only_when false;
```

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is the largest blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

